Documentación Técnica y Manual de Despliegue

Sistema Profesional de Gestión de Boutique

---

1. Tecnologías de Desarrollo

El sistema ha sido desarrollado utilizando un robusto stack tecnológico moderno orientado a la escalabilidad, seguridad y rendimiento:

- **Backend: Node.js** con el framework **Express**. Node.js permite un manejo asíncrono y veloz de las peticiones, mientras que Express facilita la creación de la arquitectura MVC (Modelo-Vista-Controlador) y las rutas de la API.
- **Base de Datos: PostgreSQL**. Un sistema de gestión de bases de datos relacional de código abierto que proporciona un alto rendimiento y confiabilidad, ideal para el manejo de transacciones, inventarios y finanzas de la tienda.
- **Frontend (Vistas): EJS (Embedded JavaScript templating)** para la generación de vistas dinámicas desde el servidor, en combinación con **HTML5, CSS3 y JavaScript** puro o frameworks de diseño como Bootstrap para lograr una estética profesional de "Boutique".
- **Seguridad:** Uso de **bcryptjs** para el encriptado de las contraseñas, protegiendo la información de los usuarios. Manejo de sesiones persistentes con `express-session` para la autenticación y autorización (Login/Logout seguro).
- **Manejo de Archivos: Multer** para la subida de recursos.
- **Exportación de Reportes: ExcelJS** y **json2csv** para la generación de reportes transaccionales y de negocio.

---

2. Funcionalidades del Sistema

El sistema ofrece una cobertura integral para la administración de la boutique, dividido en los siguientes módulos:

- **Panel de Control (Dashboard):** Un resumen gerencial con indicadores clave (ventas de hoy, cantidad de clientes, alerta de stock bajo) e ingresos recientes. Diseño enfocado en widgets para rápida lectura.
- **Gestión de Caja:** Permite aperturar y cerrar la caja por cada jornada o turno, registrando saldos iniciales, ingresos totales, egresos y calculando automáticamente el saldo esperado o los descuadres (sobrantes/faltantes).
- **Gestión de Ventas (Punto de Venta - POS):** Un módulo centralizado y rediseñado estéticamente para agregar clientes, buscar productos rápidamente o mediante código de barras, aplicar diferentes medios de pago y generar el comprobante final.
- **Gestión de Compras y Proveedores:** Registro de nuevos proveedores, ingreso de mercadería a través de compras que alimentan automáticamente el inventario (Kardex), controlando número de factura y costos.
- **Inventario (Productos y Categorías):** Control de los productos a la venta, incluyendo atributos como talla, código de barras, precio de compra y venta. Incluye stock mínimo para generar alertas automatizadas, así como la agrupación lógica por categorías.
- **Kardex de Movimientos:** Trazabilidad completa por producto de entradas, salidas (ventas) y ajustes, otorgando visibilidad total al negocio sobre los historiales de movimientos.
- **Cuentas y Clientes:** Administración de la base de clientes para poder fidelizarlos y llevar el registro en las ventas y comportamientos de compra.
- **Control de Gastos:** Registro de salidas de dinero no relacionadas directamente a mercadería (ej. servicios, papelería, salarios), que repercutirán en el Cierre de Caja.
- **Reportes:** Generación de resúmenes de rendimiento del negocio durante intervalos de tiempo, exportables a formatos compartibles como Excel o CSV.
- **Seguridad y Usuarios:** Creación de usuarios con distintos roles (ej. Administrador vs. Cajero), permitiendo restringir el acceso a zonas críticas como configuraciones y reportes.

---

3. Instalación y Configuración (Computador Nuevo)

Para ejecutar esta aplicación en un ambiente local nuevo (Windows, Mac o Linux), se requieren los siguientes pasos:

3.1. Requisitos Previos

1. **Node.js**: Descargar e instalar la versión LTS (recomendado 18.x o superior) desde `https://nodejs.org/`.
2. **PostgreSQL**: Instalar el motor de base de datos desde `https://www.postgresql.org/download/`. Durante la instalación asigne una contraseña al súper usuario `postgres` (ejemplo `root` o similar).
3. **Git** (Opcional): Para clonar o descargar el repositorio.

3.2. Configuración de Base de Datos

1. Abra pgAdmin o la herramienta de línea de comandos `psql`.
2. Cree una base de datos vacía nombrada `boutique_db`.
3. Si lo hace por consola: `CREATE DATABASE boutique_db;`

3.3. Configuración del Proyecto

1. Copie o clone la carpeta del proyecto `boutique-system` en su máquina local.
2. Abra la terminal en la raíz de esta carpeta.
3. Cree un archivo `.env` en la raíz (si no existe) y coloque sus variables de entorno basadas en su instalación de PostgreSQL:

```
PORT=3000
DB_USER=postgres
DB_PASSWORD=root
DB_PORT=5432
DB_NAME=boutique_db

# Configuración Regional por Defecto
APP_LOCALE=es-PE
APP_TIMEZONE=America/Lima
APP_CURRENCY=PEN
```

(Asegúrese de modificar `DB_PASSWORD` por la contraseña que puso al instalar su PostgreSQL).

4. Instale las dependencias del proyecto ejecutando: `npm install`
5. Ejecute el script de inicialización y poblado de la base de datos: `npm run init-db` *Este script creará las tablas necesarias y puede crear un usuario administrador base.*

---

4. Ejecución de la Aplicación (CMD)

Una vez completado el paso anterior, siga estas instrucciones diariamente para levantar el sistema:

1. Abra el Símbolo del Sistema (CMD) o PowerShell en Windows.
2. Navegue hasta el directorio del sistema utilizando el comando `cd`. (Ej. `cd C:\Webs\Javascript\boutique-system`).
3. Ejecute el siguiente comando para iniciar la aplicación: `npm start`
4. Verá el mensaje: `🚀 Servidor corriendo en puerto 3000`.
5. Abra su navegador web (Chrome, Edge, etc.) y diríjase a: **`http://localhost:3000`**

---

5. Credenciales de Acceso Administrador

El sistema viene pre-configurado mediante script con una cuenta maestra con privilegios totales. Proteja esta información cuidadosamente:

> **Usuario Correo:** `admin@boutique.com`
> **Contraseña:** `admin123`
> **Rol Asignado:** Administrador (Acceso Total)

Nota: Se recomienda cambiar esta contraseña desde el módulo de Usuarios / Perfil tan pronto ingrese por primera vez en un entorno de producción.

---

6. Hosting: Recomendaciones y Guía Paso a Paso

Al ser una aplicación **Node.js con PostgreSQL**, no sirve cualquier hosting básico (como los paneles cPanel estándar orientados a PHP). Se necesitan servidores Cloud, VPS o servicios Platform-as-a-Service (PaaS).

Opciones Ideales:

- **Virtual Private Server (VPS)**: DigitalOcean, Linode, AWS EC2. (Económico, requiere conocimiento de sistemas Linux y Nginx).
- **PaaS (Platform as a Service)**: Render, Railway, Vercel, o Heroku. (Fáciles, depliegues automáticos, incluyen bases de datos gestionadas).

🏆 Sugerencia Óptima: RENDER.COM

Por qué: Render es altamente intuitivo, se integra maravillosamente con repositorios (GitHub/GitLab), provee certificados SSL automáticos y permite alojar tanto el servicio web Node.js como la base de datos PostgreSQL en una misma plataforma bajo un esquema amigable y escalable.

📝 Paso a Paso para Desplegar en Render:

Paso 1: Preparación del Repositorio

1. Suba su proyecto a un repositorio privado en GitHub. (Excepto la carpeta `node_modules` y el archivo `.env`).

Paso 2: Creación de PostgreSQL Gestionado (En Render)

1. Ingrese a `https://render.com/` y cree una cuenta (puede usar GitHub).
2. Haga clic en el botón de **"New +"** y seleccione **"PostgreSQL"**.
3. Póngale un nombre a su base de datos (e.g., `boutique-db-prod`), escoja la región más cercana y el plan (Puede ser Free o el plan pago más económico para boutiques serias).
4. Clic en "Create Database". Render le dará inmediatamente todos los datos de conexión internos (Internal Database URL) y externos.
5. Copie el campo `Internal Database URL`.

Paso 3: Creación de la Aplicación Web

1. Nuevamente haga clic en **"New +"** y seleccione **"Web Service"**.
2. Conecte su cuenta de GitHub y seleccione el repositorio de la Boutique.
3. Configure los datos básicos del Web Service:
    - **Name:** `boutique-system-app`

- **Environment:** Node
- **Build Command:** `npm install`
- **Start Command:** `npm start`

4. Ubique la sección **"Environment Variables"** (Variables de entorno) más abajo. Aquí insertará sus credenciales seguras de conexión (en base a la BD que acaba de crear):
   - `PORT` = `3000`
   - Reemplace las variables locales de BD (`DB_USER` , `DB_PASSWORD` , etc.) por la conexión brindada por Render, o agregue `DATABASE_URL` = (Pegue la *Internal Database URL* del paso anterior). *(Nota: Si usa connection string, quizás necesite un leve ajuste en su `database.js` del código base, agregando el soporte de URL)*. Añadir las variables de regionalización `APP_LOCALE` y demás.
5. Seleccione la capa de pago (e.g., Free o Starter).
6. Haga clic en **"Create Web Service"**.

Paso 4: Estructuración Inicial de Datos Una vez Render termine el despliegue (saldrá "Live" en verde):

1. Desde el panel lateral izquierdo del Web Service en Render, busque la opción **"Shell"** (Terminal remota).
2. Ejecute el comando interno: `npm run init-db` para configurar sus esquemas PostgreSQL desde la nube.

Paso 5: Acceso y DNS Render y su sistema ya estarán funcionando en línea en un subdominio parecido a `https://boutique-system-app.onrender.com` . Si adquirió un dominio personalizado (`diboutique.com`), puede agregarlo fácilmente desde la pestaña "Settings" > "Custom Domains" en Render, siguiendo las indicaciones de apuntar sus registros DNS a las direcciones que Render le otorga.